JavaScript Full Handbook

PART 1

1. Introduction to JavaScript

★ What is JavaScript ?

A programming language used to make web pages interactive.
It adds behavior, logic and dynamic content to websites.

HTML	CSS	JavaScript
Structure of web page.	Styling and layout.	Logic and interaction.

★ Why use JavaScript ?

- Runs directly in the browser.
- Used in frontend and backend development.
- Helps build interactive UI, forms, apps and games.
- Large ecosystem and community support.

★ Where JavaScript Runs ?

- Browser : Chrome, Edge, Safari, Firefox
- Server : Node.js
- Mobile/Desktop apps using frameworks.

Example :

```
console.log("Hello JavaScript");
```

★ Key Features

- Lightweight
- Event driven
- Supports functions and objects
- Supports asynchronous programming
- Works with APIs and DOM.

Note :

JavaScript is one of the core technologies of the web.

Key Point :

✓ HTML creates structure.
✓ CSS makes it beautiful.
✓ JavaScript makes it interactive.

JavaScript Full Handbook

PART 1

2. VARIABLES & DATA TYPES

★ What is a Variable ?

A variable is a container used to store data values.

★ Primitive Data Types

Primitive data types store simple values.

String	→ "JavaScript"
Number	→ 100
Boolean	→ true / false
Undefined	→ value not assigned
Null	→ empty value
BigInt	→ large numbers
Symbol	→ unique value

★ typeof Operator

We can use typeof to find the data type of a value.

```
typeof "Hello"      → "string"
typeof 10           → "number"
typeof true         → "boolean"
typeof undefined    → "undefined"
```

★ var, let and const

Keyword	Scope	Reassign	Redeclare
var	Function	Yes	Yes
let	Block	Yes	No
const	Block	No	No

```
var city = "Delhi";
let age = 25;
const country = "India";
```

★ Non-Primitive Data Types

Non-primitive data types store complex data structures.

Object	→ collection of key-value pairs
Array	→ ordered collection of values
Function	→ block of code (also an object)

Note :

- Use const by default.
- Use let only when value needs to change.

Key Point :

✓ let and const are modern JS.
✓ Avoid var in new code.
✓ Arrays and objects are reference types.

JavaScript Full Handbook

PART 1

3. OPERATORS IN JAVASCRIPT

★ What is an Operator ?

An operator is a **symbol** used to perform an **operation** on **values**.

★ Arithmetic Operators

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 3	1.67
%	Remainder	5 % 3	2
**	Power	5 ** 3	125

Example :

```
let a = 10;
let b = 3;
console.log(a + b);   // 13
console.log(a % b);   // 1
```

★ Comparison Operators

Operator	Name	Example	Result
==	Value equal	5 == 5	true
===	Value and type equal	5 === 5	true
!=	Not equal	5 != 3	true
!==	Not equal (value or type)	5 !== "5"	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false
>=	Greater than or equal to	5 >= 5	true
<=	Less than or equal to	5 <= 3	false

== vs ===

"5" == 5 → true

"5" === 5 → false

★ Logical Operators

Operator	Name	Description	Example	Result
&&	AND	True if both conditions are true	true && false	false
||	OR	True if at least one condition is true	true || false	true
!	NOT	Reverses the boolean value	!true	false

★ Assignment Operators

Operator	Name	Example	Same as
=	Assign value	x = 5	x = 5
+=	Add and assign	x += 3	x = x + 3
-=	Subtract and assign	x -= 3	x = x - 3
*=	Multiply and assign	x *= 3	x = x * 3
/=	Divide and assign	x /= 3	x = x / 3
%=	Remainder and assign	x %= 3	x = x % 3

★ Ternary Operator

A shorthand operator for a simple if...else condition.

Syntax : `condition ? trueValue : falseValue`

Example :

```
let age = 20;
let result = age >= 18 ? "Adult" : "Minor";
```

Key Point :

- ✓ Prefer === over ==.
- ✓ Operators are used in conditions, calculations and logic.

JavaScript Full Handbook

PART 1

5. Conditional Statements

★ What are Conditions ?

Conditions help us run code based on true or false decisions.

★ if Statement

Runs code only when condition is true.

```
let marks = 85;
if (marks >= 75) {
    console.log("Good Score");
}
```

★ else if Ladder

Used when we have multiple conditions.

```
let marks = 82;
let grade;
if (marks >= 90) {
    grade = "A";
} else if (marks >= 75) {
    grade = "B";
} else {
    grade = "C";
}
console.log(grade);
```

Key Point :

- ✓ Conditions control program flow.
- ✓ Use if else for ranges.
- ✓ Use switch for exact matches.

★ if else Statement

If condition is true, first block runs.
If false, else block runs.

```
let marks = 60;
if (marks >= 75) {
    console.log("Good Score");
} else {
    console.log("Need to Improve");
}
```

★ switch Statement

Used to compare one value with many cases.

```
let day = 1;
switch (day) {
    case 1:
        console.log("Mon");
        break;
    case 2:
        console.log("Tue");
        break;
    default:
        console.log("Invalid");
}
```

Best Practices :

- Use === for comparison.
- Keep conditions simple.
- Avoid too much nesting.
- Use switch for fixed values.

JavaScript Full Handbook

PART 1

4. Type Conversion

★ What is Type Conversion ?

Changing one data type into another data type.

★ Two Types of Conversion

Type	Description
Implicit Conversion	JavaScript converts automatically.
Explicit Conversion	Developer converts manually.

★ Implicit Conversion Examples

```
"5" + 2    →  "52"
"5" - 2    →  3
true + 1   →  2
```

★ Explicit Conversion Examples

```
Number("10")   →  10
String(100)    →  "100"
Boolean(1)     →  true
```

★ Example Code

```
let x = "20";
let y = Number(x);
console.log(y + 5);   // 25
```

Note :

Unexpected conversion can create bugs in JavaScript.

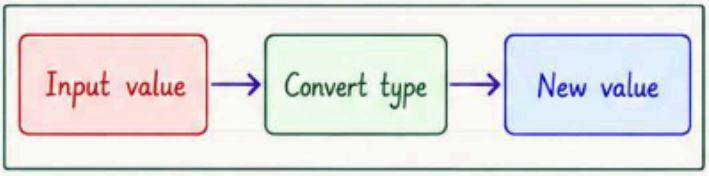
★ Truthy Values

- non-empty string
- non-zero number
- arrays and objects

★ Falsy Values

- false
- 0
- ""
- null
- undefined
- NaN

★ Conversion Process



Key Point :

- ✓ Use explicit conversion for clarity.
- ✓ Empty string, 0 and null are falsy.
- ✓ "5" + 2 becomes string.

JavaScript Full Handbook

PART 1

6. Loops in JavaScript

★ What is a Loop ?

A loop is used to repeat a block of code again and again.

★ Why use Loops ?

- Avoid repeated code.
- Process arrays and lists.
- Print patterns.
- Run checks multiple times.

★ for Loop

Used when number of iterations is known.

```
for (let i = 1; i <= 5; i++) {
    console.log(i);                # start from 1
}                                  # run till 5
                                   # output: 1 2 3 4 5
```

★ while Loop

Runs while condition is true.

```
let i = 1;                         # initialize
while (i <= 5) {                   # check condition
    console.log(i);                # print value
    i++;                           # increment
}                                  # output: 1 2 3 4 5
```

★ do while Loop

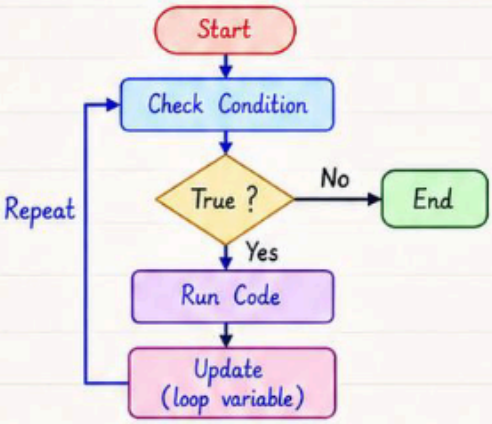
Runs at least once before checking condition.

```
let i = 1;
do {                               # print first
    console.log(i);                # then check
    i++;
} while (i <= 5);                  # output: 1 2 3 4 5
```

★ break and continue

Statement	Description	Example
break	Exits the loop immediately.	`for (let i = 1; i <= 5; i++) { if (i === 3) break; console.log(i); }` # Output: 1 2
continue	Skips the current iteration and moves to the next one.	`for (let i = 1; i <= 5; i++) { if (i === 3) continue; console.log(i); }` # Output: 1 2 4 5

★ Loop Flow Diagram



Note :

Always update loop variables to avoid infinite loops.

Key Point :

- ✓ for loop is common for counting.
- ✓ while loop is good for condition based repetition.
- ✓ break and continue control loop flow.

JavaScript Full Handbook

PART 1

7. Functions Basics

★ What is a Function ?

A function is a reusable block of code that performs a specific task.
It helps avoid repeated code and improves readability.

★ Why use Functions ?

- Reusability
- Better organization
- Easy testing and debugging
- Breaks big logic into smaller parts

★ Function Syntax

```
function functionName(parameters) {
    // block of code
    return value;
}
```

★ Function Declaration

```
function greet() {
    console.log("Hello JavaScript");
}
greet();
```

★ Parameters vs Arguments

Term	Description	Example
Parameters	Variables listed in function definition.	function add(a, b) { ... }
Arguments	Actual values passed during function call.	add(5, 3);

★ Return Statement

The return keyword sends a result back from a function.

```
function add(a, b) {
    return a + b;
}
let result = add(5, 3);
console.log(result);  // 8
```

Note :

A function runs only when it is called.

Key Point :

- ✓ Functions reduce repetition.
- ✓ Use meaningful function names.
- ✓ return gives output from function.

JavaScript Full Handbook

PART 1

8. ARROW FUNCTIONS

★ What is an Arrow Function ?

Arrow function is a shorter way to write functions in JavaScript.
It was introduced in ES6.

★ When to Use ?

- Small functions
- Callbacks
- Array methods like map, filter, reduce
- Cleaner one-line logic

★ Normal Function vs Arrow Function

Normal Function	Arrow Function
`function add(a, b) { return a + b; }`	`const add = (a, b) => { return a + b; };`

★ Short Syntax

```
const square = n => n * n;
console.log(square(5));  // 25
```

Note :
For object methods, normal functions are often safer when using this.

★ Important Difference

Arrow functions do not have their own this.
They use this from surrounding scope.

Example :

```
const greet = name => "Hello " + name;
console.log(greet("Aman"));
```

Key Point :

- ✓ Arrow functions make code shorter.
- ✓ Useful for callbacks.
- ✓ No own this binding.

JavaScript Full Handbook

PART 1

9. SCOPE IN JAVASCRIPT

★ What is Scope ?

Scope decides where a variable can be accessed in code.

★ Types of Scope

Type	Description
Global Scope	accessible everywhere
Function Scope	accessible inside function
Block Scope	accessible inside { }
Lexical Scope	inner function can access outer variables

★ Global Scope

Variables declared in global scope are accessible everywhere.

```
let name = "JS";
function show() {
        console.log(name);
}
```

★ Function Scope

Variables declared inside a function cannot be accessed outside.

```
function test() {
     let x = 10;
}
console.log(x);   // Error
```

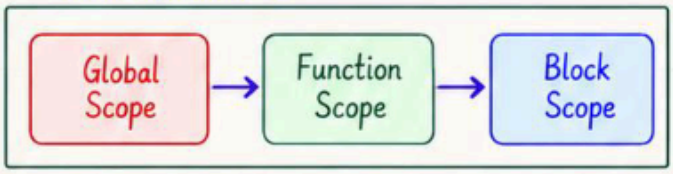
★ Block Scope

let and const follow block scope.

```
if (true) {
    let age = 25;
}
console.log(age);  // Error
```

★ Lexical Scope

Inner functions remember variables from outer functions.



Note :

var is function scoped, but let and const are block scoped.

Key Point :

- ✓ Scope protects variables.
- ✓ Prefer let and const.
- ✓ Inner scope can access outer scope.

JavaScript Full Handbook

PART 1

10. Hoisting

★ What is Hoisting ?

Hoisting means JavaScript moves declarations to the top of their scope before execution.

★ Variable Hoisting

Keyword	Hoisting Behavior
var	Hoisted and initialized with undefined.
let	Hoisted but in temporal dead zone.
const	Hoisted but in temporal dead zone.

★ var Example

```
console.log(a);  // undefined
var a = 10;
```

★ let Example

```
console.log(b);  // Error
let b = 20;
```

★ Function Hoisting

Function declarations can be called before they are defined.

```
greet();
function greet() {
    console.log("Hello");
}
```

★ Function Expression

Function expressions are not fully hoisted like declarations.

```
sayHi();  // Error
const sayHi = function() {
    console.log("Hi");
};
```

Note :

Temporal Dead Zone means variable exists but cannot be used before declaration.

Key Point :

- ✓ Function declarations are hoisted.
- ✓ Avoid using variables before declaration.
- ✓ let and const prevent many hoisting bugs.

JavaScript Full Handbook

PART 1

11. ARRAYS IN JAVASCRIPT

★ What is an Array ?

An array is an ordered collection of values.
Array index starts from 0.

★ Creating Arrays

```
let fruits = ["Apple", "Mango", "Banana"];
let numbers = [10, 20, 30];
```

★ Accessing Values

fruits[0]	→ "Apple"
fruits[1]	→ "Mango"

★ Updating Values

```
fruits[1] = "Orange";
```

★ Array Properties

length gives total number of elements.

```
console.log(fruits.length);
```

★ Common Array Methods

Method	Description
push()	Add element at the end of array.
pop()	Remove element from the end of array.
unshift()	Add element at the start of array.
shift()	Remove element from the start of array.
includes()	Checks if a value exists in the array.
indexOf()	Returns the index of a value (in -1 if not found).

★ Looping Array

```
for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}
```

Note :

Arrays can store mixed data, but same type values are easier to manage.

Key Point :

- ✓ Array index starts from 0.
- ✓ length gives size.
- ✓ Arrays are useful for lists of data.

JavaScript Full Handbook

PART 1

12. ARRAY METHODS

★ Why Array Methods ?

Array methods help process list data in a clean and readable way.

★ forEach()

Runs a function for each element.

```
numbers.forEach(n => console.log(n));
```

★ map()

Creates a new array by transforming each element.

```
let doubled = numbers.map(n => n * 2);
```

★ filter()

Creates a new array with matching values.

```
let even = numbers.filter(n => n % 2 === 0);
```

★ reduce()

Reduces array to a single value.

```
let sum = numbers.reduce((a, b) => a + b, 0);
```

★ Other Useful Methods

Method	Description
find()	returns first matching value
some()	checks if at least one matches
every()	checks if all match
sort()	sorts array
slice()	returns part of array
splice()	add/remove items

Note :

map, filter and reduce return new values and help write cleaner code.

★ map vs filter vs reduce

Method	Purpose	Example
map()	transform all values	[1, 2].map(n => n * 2) // [2, 4]
filter()	select values	[1, 2].filter(n => n > 1) // [2]
reduce()	calculate one result	[1, 2].reduce((a, b) => a + b, 0) // 3

Key Point :

- ✓ forEach is for iteration.
- ✓ map returns new array.
- ✓ reduce returns single result.

JavaScript Full Handbook

@aman_views

13. OBJECTS IN JAVASCRIPT

★ What is an Object ?

An object stores data in key-value pairs.
It is used to represent real-world entities.

Term	Description
Property	data value
Method	function behavior
Object	collection of properties

★ Object Syntax

```
let user = {
    name: "Aman",
    age: 25,
    city: "Delhi"
};
```

★ Accessing Properties

Dot notation :

```
user.name
```

Bracket notation :

```
user["age"]
```

★ Updating Properties

```
user.age = 26;
user.country = "India";
```

★ Object Methods

A function inside an object is called a method.

```
let person = {
    name: "Aman",
    greet: function() {
        console.log("Hello");
    }
};
```

★ Nested Object

```
let student = {
    name: "Raj",
    marks: {
        math: 90,
        science: 85
    }
};
```

Note :
Objects are very important for APIs, JSON and real-world data.

Key Point :

- ✓ Objects store key-value data.
- ✓ Dot notation is commonly used.
- ✓ Methods define object behavior.

JavaScript Full Handbook

@aman_views

14. DESTRUCTURING & SPREAD

★ What is Destructuring ?

Destructuring helps unpack values from arrays or objects into variables.

★ Array Destructuring

```
let colors = ["red", "green", "blue"];
let [first, second] = colors;
console.log(first);  // red
```

★ Object Destructuring

```
let user = { name: "Aman", age: 25 };
let { name, age } = user;
```

★ Default Values

```
let { city = "Delhi" } = user;
```

Note :

Rest and spread use the same `...` syntax but work differently.

★ Comparison

Concept	Purpose
Destructuring	unpack values
Rest	collect remaining
Spread	expand/copy values

★ Rest Operator

Collects remaining values.

```
let [a, ...rest] = [1, 2, 3, 4];
// rest = [2, 3, 4]
```

★ Spread Operator

Expands arrays or objects.

```
let arr1 = [1, 2];
let arr2 = [...arr1, 3, 4];
```

★ Spread with Objects

```
let newUser = { ...user, city: "Delhi" };
```

Key Point :

- ✓ Destructuring makes code clean.
- ✓ Spread copies and merges data.
- ✓ Rest collects remaining values.

JavaScript Full Handbook

@aman_views

15. STRINGS IN JAVASCRIPT

★ What is a String ?

A string is a sequence of characters.
It can be written using single quotes, double quotes or backtick quotes.

```
let msg = "Hello JavaScript";
```

★ String Indexing

Each character has an index starting from 0.

J	a	v	a
0	1	2	3

★ Template Literals

Used with backtick quotes to insert variables.

```
let name = "Aman";
let text = "Hello " + name; // same idea
                           // as template literal
```

★ String Immutability

Strings cannot be changed directly.
Any modification creates a new string.

```
let s = "Java";
s[0] = "K"; // not changed
```

★ Useful Example

```
let email = " test@gmail.com ";
let clean = email.trim().toLowerCase();
```

★ Common String Methods

Method	Description
length	returns size of the string
toUpperCase()	converts to uppercase
toLowerCase()	converts to lowercase
trim()	removes spaces from start/end
includes()	checks if text exists
replace()	replaces text with new text
slice()	extracts part of the string
split()	converts string to an array

Note :

Strings are case-sensitive in JavaScript.

Key Point :

- ✓ String index starts from 0.
- ✓ Template literals are powerful.
- ✓ String methods return new values.

JavaScript Full Handbook

@aman_views

16. DATE & MATH OBJECTS

★ Date Object

Date object is used to work with date and time.

```
let now = new Date();
console.log(now);
```

★ Date Example

```
let d = new Date();
let year = d.getFullYear();
```

★ Math Object

Math object provides useful mathematical methods.

★ Common Math Methods

Method	Description
Math.round()	nearest integer
Math.floor()	lower integer
Math.ceil()	upper integer
Math.max()	maximum
Math.min()	minimum
Math.random()	random number
Math.sqrt()	square root
Math.pow()	power

★ Common Date Methods

Method	Description
getFullYear()	year
getMonth()	month index 0 - 11
getDate()	day of month
getDay()	day of week
getHours()	hours
getMinutes()	minutes

Note :

Month starts from 0 in JavaScript Date.

★ Random Number Example

```
let num = Math.floor(Math.random() * 10) + 1;
// 1 to 10
```

★ Random 1 to N

Random 1 to N :

```
Math.floor(Math.random() * N) + 1
```

Key Point :

- ✓ Date handles time values.
- ✓ Math.random() gives 0 to less than 1.
- ✓ Math.floor() is useful for random integers.

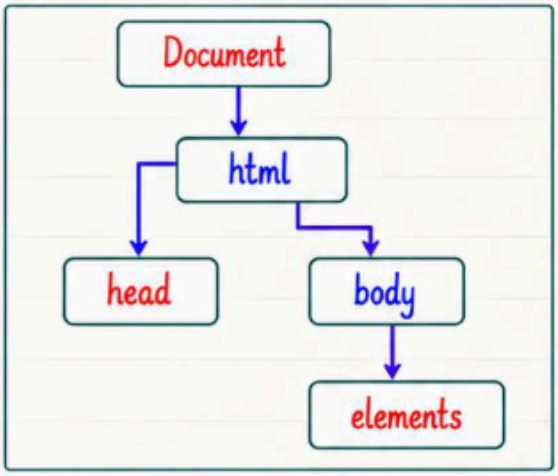
JavaScript Full Handbook

@aman_views

17. DOM BASICS

★ What is DOM ?

DOM stands for Document Object Model.
It represents HTML page as a tree of objects.



★ Why DOM is Used ?

- Read HTML elements
- Change text and style
- Add or remove elements
- Build interactive web pages

★ Selecting Elements

Method	Description
getElementById()	select by id
getElementsByClassName()	select by class
getElementsByTagName()	select by tag
querySelector()	first matching element
querySelectorAll()	all matching elements

★ Changing Text

```
document.getElementById("title").innerText = "Hello";
```

★ Changing Style

```
let box = document.querySelector(".box");
box.style.color = "red";
```

★ Changing HTML

```
box.innerHTML = "<b>Updated</b>";
```

★ Creating Element

```
let p = document.createElement("p");
p.innerText = "New paragraph";
```

Note :

DOM connects JavaScript with HTML page.

Key Point :

- ✓ DOM is a tree structure.
- ✓ querySelector is commonly used.
- ✓ JS can change page content dynamically.

JavaScript Full Handbook

@aman_views

18. DOM EVENTS

★ What is an Event ?

An event is an action that happens on a web page.

Example : click, input, submit, mouseover.

★ Event Listener

Used to run code when an event happens.

```
button.addEventListener("click", function() {
    console.log("Button clicked");
});
```

★ Common Events

Event	Description
click	user clicks element
input	user types in input field
submit	form submitted
change	value changed
mouseover	mouse enters element
keydown	keyboard key pressed

★ Event Object

Event object gives details about the event.

```
input.addEventListener("input", function(e) {
    console.log(e.target.value);
});
```

★ Prevent Default

Stops default browser behavior.

```
form.addEventListener("submit", function(e) {
    e.preventDefault();
});
```

★ Event Flow

User Action → Event Trigger → Listener Runs → Code Executes

★ Real Use Cases

- Button click actions
- Form validation
- Search box filtering
- Dropdown handling

Note :

Events make web pages interactive and dynamic.

Key Point :

- ✓ addEventListener is the modern way.
- ✓ Event object gives event details.
- ✓ preventDefault stops default action.